SIMATS ENGINEERING

ASSESSMENT TOOL 2: Error Analysis Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Register Number	192521391
Name	BRIJESH.T

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.* (BL5)

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 25**

Question Count: 5

Code of Conduct

I __BRIJESH.T/192521391__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____ _____

Assessment Tasks

Error Analysis Task

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.
2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.
3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.
4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.
5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Error Analysis Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Identification of Error	30%	Accurately identifies the root technical issue	Identifies most of the issue	Partially identifies issue	Fails to identify issue
Cause Analysis	20%	Explains root cause with strong technical reasoning	Reasonable cause analysis	Basic cause analysis	Weak analysis
Corrective Action	25%	Proposes effective and feasible corrections	Reasonable corrections	Basic corrections	Ineffective corrections
Use of Hadoop / Integration Concepts	15%	Applies relevant concepts appropriately	Mostly appropriate application	Partial application	Incorrect application
Clarity	10%	Clear and direct explanation	Generally clear	Somewhat unclear	Poorly presented

1. Hadoop Job Fails Because Input Splits Are Uneven

Problem:
The input data is divided into uneven splits. Some Mapper tasks get more data while others finish quickly. Because of this, some Reducers remain idle.

Likely design error:

- Improper input split size.
- Data is not evenly distributed.
- Poor partitioning of intermediate data.
- Too few Mapper or Reducer tasks.

Corrective actions:

- Use a suitable HDFS block and input split size.
- Balance the input files across DataNodes.
- Use a proper partitioner to distribute keys evenly.
- Set a suitable number of Reducers.
- Use a Combiner to reduce unnecessary network traffic.

Result:
Balanced input and better partitioning will keep Mappers and Reducers busy and reduce the total job time.

2. HDFS Data Becomes Unavailable After One Node Failure

Problem:
If data becomes unavailable after only one DataNode fails, the HDFS system is probably not properly replicated or does not have a highly available NameNode setup.

Likely design error:

- Replication factor may be set to **1**.
- Replicas may not be distributed across different DataNodes.
- NameNode may be a single point of failure.
- Some files may have missing or under-replicated blocks.

Corrective actions:

- Set the replication factor to **3** for important data.
- Store replicas on different DataNodes.
- Use **Active and Standby NameNodes**.
- Regularly monitor under-replicated blocks.
- Run HDFS rebalancing when required.

Result:
Even if one DataNode fails, another DataNode can provide the required data.

3. ETL and NiFi Create Duplicate Records

Problem:
The same data may be processed more than once before reaching the analytics database.

Likely causes:

- NiFi flow may retry a record after a failure without checking whether it was already stored.
- ETL process may repeatedly read the same source records.
- No unique ID is assigned to each record.
- Multiple ingestion flows may process the same data.
- Poor handling of checkpoints or timestamps.

Corrective actions:

- Give every record a **unique ID**.
- Use incremental loading instead of loading the complete dataset repeatedly.
- Maintain checkpoints or last-processed timestamps.
- Use NiFi deduplication techniques.
- Add database **unique constraints**.
- Use upsert/merge operations instead of blindly inserting records.

Result:
These methods make the ingestion process more reliable and prevent duplicate data.

4. YARN Resource Starvation

Problem:
When many users submit jobs together, one or more jobs may consume most of the available CPU and memory. Other jobs then wait for resources.

Likely scheduling error:

- No proper resource limits are configured.
- All users may be using the same queue.
- No priority system is defined.
- One large job can consume most cluster resources.

Corrective actions:

- Use **YARN Capacity Scheduler or Fair Scheduler**.
- Create separate queues for different users or workloads.
- Set minimum and maximum resource limits for queues.
- Give important jobs higher priority.
- Allocate CPU and memory fairly between users.
- Monitor cluster resource usage.

Result:
Resources are shared fairly, preventing one user or job from starving the rest of the cluster.

5. Backup Restores Outdated Data

Problem:
After recovery, the system restores an older version of the data instead of the latest version.

Likely design flaw:

- Backups are not performed frequently enough.
- Only one old backup copy is maintained.
- Replication is being treated as a backup.
- Backup checkpoints are not updated correctly.
- Recovery is using the wrong backup version.

Corrective actions:

- Schedule regular and automatic backups.
- Maintain multiple backup versions.
- Use incremental backups along with periodic full backups.
- Store backups separately from the main system.
- Maintain timestamps and backup version information.
- Test the restore process regularly.

Result:
A proper versioned backup strategy ensures that the latest valid data can be recovered after a failure.